

KONGU ENGINEERING COLLEGE

GPS -BASED VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM

MECHATRONICS AND IOT

EXCERSICE – 15

HEMAVARSHINI S – 24MER017

KAVIN KUMARAN S – 24MER023

KOWSIKA K – 24MER026

VISHNUPRASATH B – 24MER062



KONGU ENGINEERING COLLEGE

— TRANSFORM YOURSELF —

GPS -BASED VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM

PROJECT OVERVIEW

The GPS-Based Vehicle Tracking and Fleet Management System is an IoT-based transportation monitoring system developed to track and manage vehicles in real time. The system uses a GPS module to continuously obtain the vehicle's geographical position, including latitude, longitude, speed, and movement status.

A microcontroller such as ESP32 or NodeMCU processes the GPS information and transmits the collected data to a cloud platform through Wi-Fi or GSM communication. The fleet operator can access the information through a web or mobile dashboard and monitor the current location of vehicles remotely.

The system provides important fleet-management functions such as real-time location tracking, route monitoring, speed monitoring, trip history, vehicle status monitoring, and geofencing alerts. Historical GPS data can also be stored for analyzing vehicle routes, travel time, and fleet performance.

By providing continuous visibility of vehicle movement, the system helps organizations reduce unnecessary travel, improve route planning, minimize fuel consumption, increase vehicle utilization, and improve transportation safety.

The overall system follows the process: GPS Satellite → GPS Module → ESP32/NodeMCU → Wi-Fi/GSM → Cloud Server → Fleet Dashboard → Fleet Operator.

The proposed system provides a smart, reliable, and efficient solution for modern vehicle tracking and fleet management, making transportation operations more transparent, secure, and cost-effective.

PROBLEM STATEMENT

Traditional vehicle tracking methods depend heavily on manual monitoring and provide limited information about the vehicle's current location and operating condition. Fleet operators may face difficulties in monitoring multiple vehicles, tracking routes, identifying delays, and maintaining accurate travel records.

The major problems include:

- Lack of real-time vehicle location tracking.
- Difficulty in monitoring multiple vehicles simultaneously.
- Inefficient route planning and management.
- Increased fuel consumption due to unnecessary travel.
- Difficulty in identifying unauthorized routes or vehicle usage.
- Limited information about vehicle speed and movement.
- Manual maintenance of vehicle travel records.
- Delayed response to vehicle-related problems or emergencies.
- Difficulty in analyzing historical vehicle routes and performance.
- Lack of centralized fleet monitoring and management.

To overcome these challenges, a GPS-based vehicle track and fleet management system is proposed. The system continuously collects GPS location and vehicle movement data and transmits it to a cloud-based monitoring platform. Fleet operators can remotely view vehicle locations, monitor routes, analyze travel information, and improve overall fleet efficiency.

The proposed system helps reduce operational costs, improve vehicle utilization, enhance transportation safety, and provide better control over fleet operations.

PROBLEM SOLUTION

The proposed GPS-Based Vehicle Tracking and Fleet Management System provides an automated solution for monitoring and managing vehicles in real time. GPS technology is used to determine the exact location of the vehicle, while a microcontroller processes and transmits the collected information to a cloud-based monitoring platform.

The main solutions provided by the system are:

- Real-time tracking of vehicle location using GPS.
- Continuous monitoring of latitude, longitude, and vehicle speed.
- Wireless transmission of vehicle data to the cloud server.
- Remote monitoring of vehicles through a dashboard.
- Monitoring of multiple vehicles from a centralized platform.
- Recording and analysis of vehicle routes and travel history.
- Geofencing to identify unauthorized movement or route deviation.
- Improved route planning to reduce travel time and fuel consumption.
- Faster identification of delays and abnormal vehicle movement.
- Better fleet utilization and transportation management.
- Reduced dependence on manual vehicle monitoring.
- Improved vehicle safety through continuous tracking and alerts.

The system provides fleet operators with accurate and continuously updated vehicle information. This enables better decision-making, efficient route management, reduced operational costs, and improved overall transportation performance.

Overall, the proposed system offers a reliable, automated, and cost-effective solution for modern fleet tracking and vehicle management.

SYSTEM ARCHITECTURE

The GPS-Based Vehicle Tracking and Fleet Management System consists of several interconnected hardware and software modules that work together to collect, process, transmit, store, and display vehicle information.

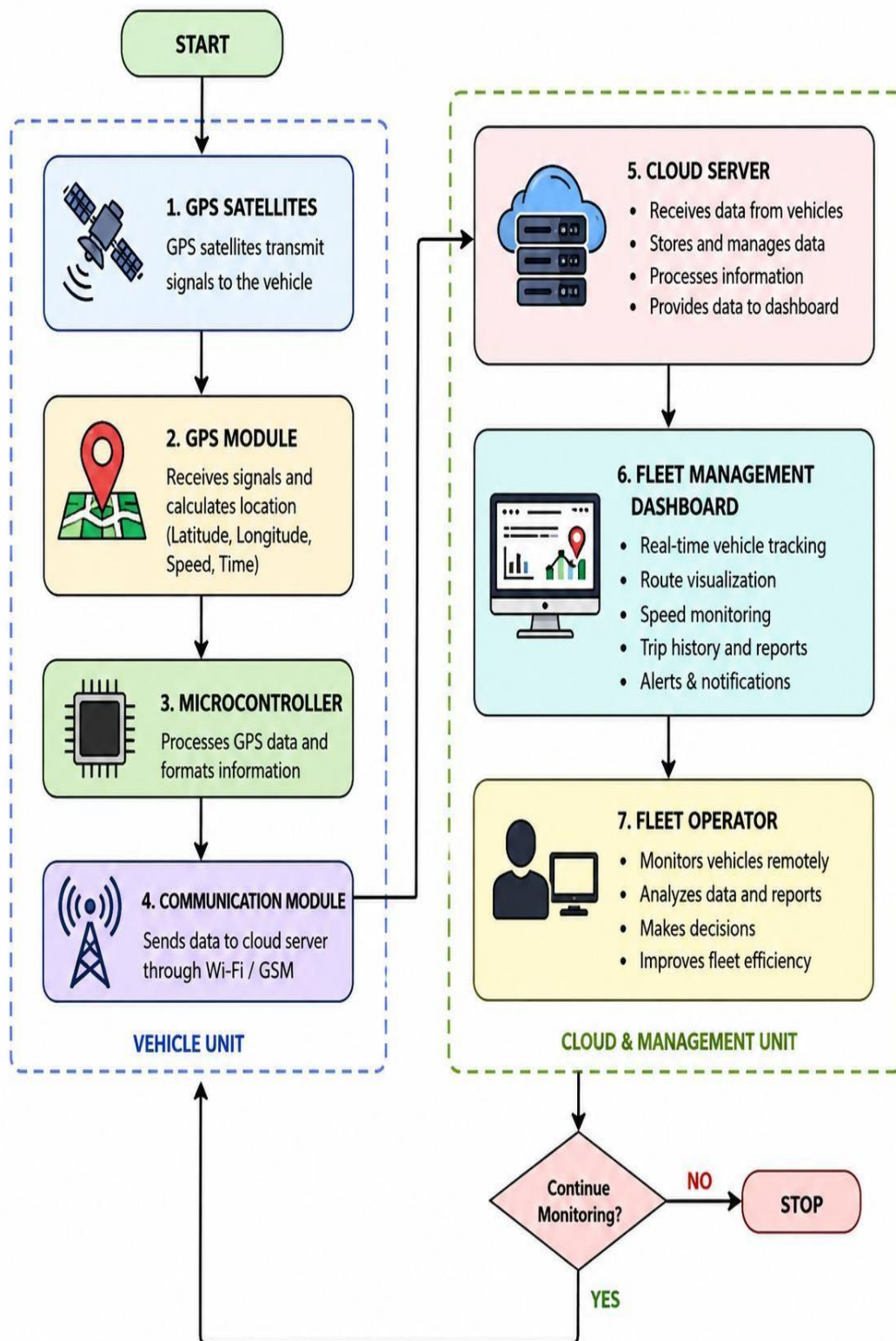
The GPS module receives signals from GPS satellites and determines the vehicle's current latitude, longitude, speed, and movement information. The microcontroller, such as ESP32 or NodeMCU, receives the GPS data, processes it, and prepares it for wireless transmission.

The processed data is transmitted through Wi-Fi or GSM communication to a cloud server. The cloud platform stores the vehicle information and makes it available to the fleet management dashboard. The fleet operator can access the dashboard remotely to monitor vehicle location, routes, speed, and travel history.

The system architecture consists of the following major sections:

- GPS Satellite: Provides positioning signals required to determine the vehicle's geographical location.
- GPS Module: Receives satellite signals and calculates latitude, longitude, speed, and other positioning data.
- Microcontroller: Processes GPS information and controls communication between the vehicle unit and cloud platform.
- Communication Module: Transfers vehicle data to the cloud using Wi-Fi or GSM.
- Cloud Server: Receives, stores, and manages vehicle tracking information.
- Fleet Management Dashboard: Displays real-time vehicle locations, routes, speed, and historical information.
- Fleet Operator: Monitors vehicles remotely and uses the collected information for transportation management.

SYSTEM ARCHITECTURE FLOW CHART



The overall data flow is:

GPS Satellite → GPS Module → ESP32/NodeMCU → Wi-Fi/GSM → Cloud Server → Fleet Management Dashboard → Fleet Operator

The architecture enables continuous data communication between the vehicle and the monitoring platform, providing real-time visibility and centralized control of fleet operations.

CIRCUIT TABLE

COMPONENT	QUANTITY	CONNECTION
ESP8266 NodeMCU	1	-
NEO-6M GPS MODULE	1	TX->D5, RX->D6
OLED DISPLAY	1	SDA->D2, SCL->D1
Li-ion BATTERY		
TP4056 CHARGING MODULE	1	BATTERY TERMINALS
JUMPER WIRES	1 Set	ALL MODULES

The circuit consists of an ESP8266 NodeMCU as the main controller, a NEO-6M GPS module for obtaining the vehicle’s real-time location, and an OLED display for showing the tracking information. The GPS module receives signals from satellites and sends latitude, longitude, speed, and location data to the NodeMCU through serial communication.

The OLED display is connected to the NodeMCU using the I²C interface and provides a local display of GPS coordinates and vehicle status. A Li-ion battery is used as the primary power source, while the TP4056 charging module provides safe and convenient battery charging. Jumper wires are used to establish electrical connections between the components.

The NodeMCU processes the GPS data and can transmit the information through its built-in Wi-Fi capability to a cloud platform or fleet monitoring dashboard. This allows the vehicle location to be monitored remotely while also providing local information through the OLED display. The complete circuit provides a compact and portable solution for real-time vehicle tracking and fleet management.

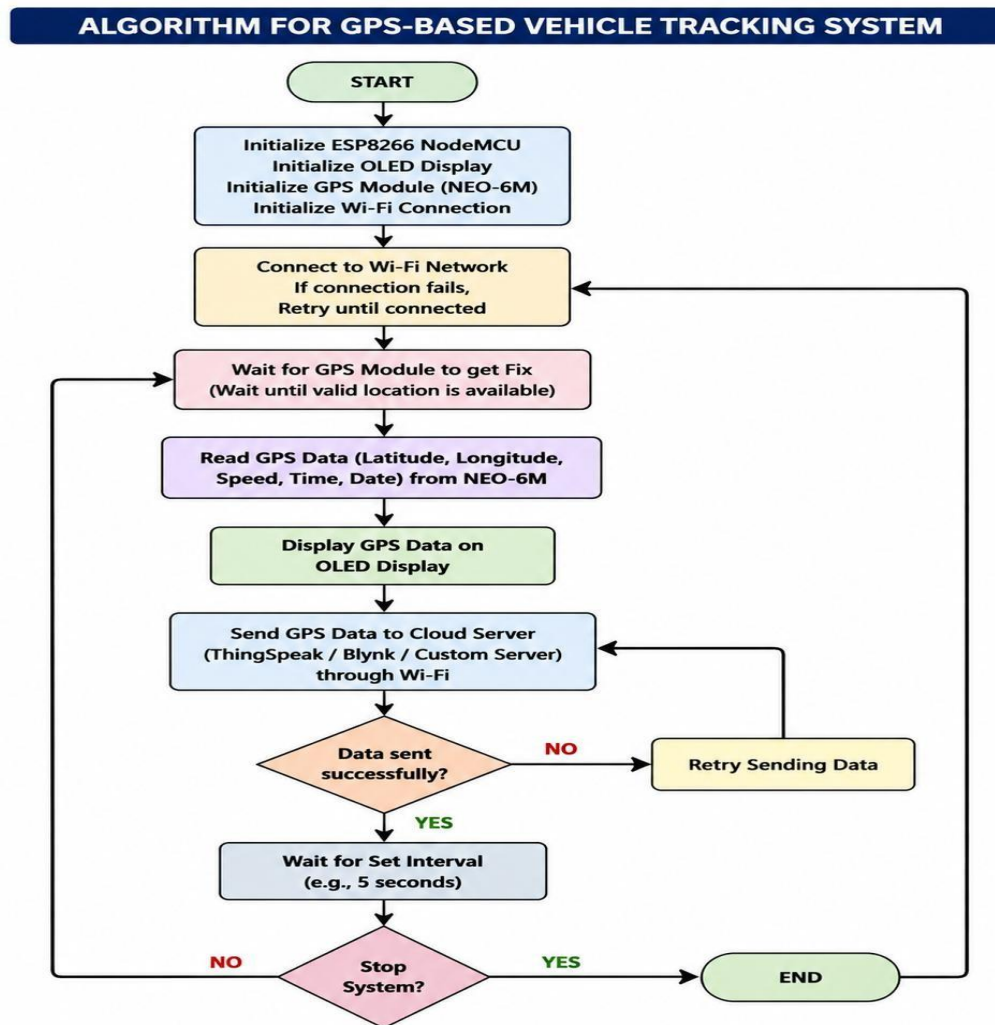
CIRCUIT DESIGN

The circuit is designed around the ESP8266 NodeMCU, which acts as the main controller of the GPS-based vehicle tracking system. The NEO-6M GPS module is connected to the NodeMCU through serial communication to receive real-time vehicle location data such as latitude, longitude, speed, and movement information.

The OLED display is connected using the I²C interface and displays the GPS coordinates and vehicle status. The Li-ion battery supplies power to the system through the TP4056 charging module, which is used for battery charging and power management. Jumper wires are used to establish the connections between the components.

The overall circuit provides a compact and portable tracking unit that collects GPS data, processes it using the NodeMCU, displays important information on the OLED, and enables wireless transmission for remote fleet monitoring.

connection is established. Once connected, the system starts receiving signals from GPS satellites and waits until the GPS module obtains a valid location fix.



When a valid GPS signal is available, the NodeMCU reads important vehicle information such as latitude, longitude, speed, date, and time. The collected GPS information is processed and displayed on the OLED screen so that the vehicle operator can view the current tracking details locally.

This process continues repeatedly to provide continuous vehicle tracking and updated fleet information. The system can therefore provide real-time location monitoring, speed information, route tracking, and historical data collection. The tracking cycle continues until the system is

manually stopped or powered off, providing an automated and reliable method for monitoring vehicle movement.

- Start the system.
- Initialize ESP8266 NodeMCU, NEO-6M GPS module, OLED display, Wi-Fi, and required communication.
- Connect the ESP8266 to the Wi-Fi network.
- Initialize communication with the GPS module.
- Wait until the GPS module obtains a valid satellite signal.
- Read latitude, longitude, speed, date, and time from the GPS module.
- Process and validate the received GPS data.
- Display the vehicle location and status on the OLED display.
- Transmit the GPS data through Wi-Fi to the cloud server.
- Store and visualize the vehicle tracking information on the monitoring platform.
- Check whether the data transmission is successful.
- If transmission fails, retry sending the data.
- If transmission is successful, wait for the predefined time interval.
- Continue tracking and repeat the process continuously.

CODING

```
// =====  
// GPS-BASED VEHICLE TRACKING AND FLEET MANAGEMENT SYSTEM  
// ESP32 + NEO-6M GPS + OLED + Adafruit IO  
// =====  
  
#include <WiFi.h>  
#include <TinyGPSPlus.h>  
#include <Wire.h>  
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#include "AdafruitIO_WiFi.h"
```

```
// =====
```

```
// 1. WIFI DETAILS
```

```
// =====
```

```
#define WIFI_SSID "IQOO Z10 5G"
```

```
#define WIFI_PASSWORD "gemcvdbr4vk8wbx"
```

```
// =====
```

```
// 2. ADAFRUIT IO DETAILS
```

```
// =====
```

```
#define IO_USERNAME "Vishnuprasath_B"
```

```
#define IO_KEY "aio_iyDk23M0tgQZmlRTNevwApeuCmsQ"
```

```
AdafruitIO_WiFi io(
```

```
  IO_USERNAME,
```

```
  IO_KEY,
```

```
  WIFI_SSID,
```

```
  WIFI_PASSWORD
```

```
);
```

```
// =====
```

```
// 3. ADAFRUIT IO FEEDS
```

```
// =====
```

```
AdafruitIO_Feed *latitudeFeed =
```

```
  io.feed("latitude");
```

```
AdafruitIO_Feed *longitudeFeed =  
  io.feed("longitude");
```

```
AdafruitIO_Feed *speedFeed =  
  io.feed("speed");
```

```
AdafruitIO_Feed *locationFeed =  
  io.feed("location");
```

```
AdafruitIO_Feed *statusFeed =  
  io.feed("status");
```

```
// =====  
// 4. GPS  
// =====
```

```
#define GPS_RX 16  
#define GPS_TX 17
```

```
HardwareSerial GPS_Serial(2);
```

```
TinyGPSPlus gps;
```

```
// =====  
// 5. OLED  
// =====
```

```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64
```

```
#define OLED_SDA 21
```

```
#define OLED_SCL 22
```

```
#define OLED_ADDRESS 0x3C
```

```
Adafruit_SSD1306 display(
```

```
    SCREEN_WIDTH,
```

```
    SCREEN_HEIGHT,
```

```
    &Wire,
```

```
    -1
```

```
);
```

```
// =====
```

```
// 6. TIMER
```

```
// =====
```

```
unsigned long previousMillis = 0;
```

```
const unsigned long UPDATE_INTERVAL = 10000;
```

```
// =====
```

```
// 7. OLED DISPLAY
```

```
// =====
```

```
void updateOLED(
```

```
    double latitude,
```

```
    double longitude,
```

```
    double speed,
```

```
    bool gpsValid
```

```
)  
{  
  display.clearDisplay();  
  
  display.setTextColor(  
    SSD1306_WHITE  
  );  
  
  // -----  
  // TITLE  
  // -----  
  
  display.setTextSize(1);  
  
  display.setCursor(10, 0);  
  
  display.println(  
    "VEHICLE TRACKER"  
  );  
  
  // -----  
  // GPS DATA  
  // -----  
  
  if (gpsValid)  
  {  
    display.setCursor(0, 15);  
  
    display.print("LAT:");
```

```
display.println(  
    latitude,  
    5  
);
```

```
display.setCursor(0, 28);
```

```
display.print("LON:");
```

```
display.println(  
    longitude,  
    5  
);
```

```
display.setCursor(0, 41);
```

```
display.print("SPD:");
```

```
display.print(  
    speed,  
    1  
);
```

```
display.println(" km/h");
```

```
display.setCursor(0, 54);
```

```
display.print(  
    "GPS: ACTIVE"  
);
```



```
}

else
{
    display.setCursor(15, 25);

    display.println(
        "GPS SEARCHING"
    );

    display.setCursor(20, 40);

    display.println(
        "WAITING..."
    );
}

display.display();
}

// =====
// 8. CONNECT TO ADAFRUIT IO
// =====

void connectAdafruitIO()
{
    Serial.println();
    Serial.println(
        "Connecting to Adafruit IO..."
    );
}
```

```
io.connect();

while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}

Serial.println();

Serial.println(
  "Adafruit IO Connected!"
);

Serial.print(
  "Status: "
);

Serial.println(
  io.statusText()
);

statusFeed->save(
  "System Online"
);
}

// =====
// 9. SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200);
```

```
  delay(1000);
```

```
  Serial.println();
```

```
  Serial.println(
```

```
    "=====
```

```
);
```

```
  Serial.println(
```

```
    " GPS VEHICLE TRACKING SYSTEM"
```

```
);
```

```
  Serial.println(
```

```
    "=====
```

```
);
```

```
// =====
```

```
// GPS
```

```
// =====
```

```
GPS_Serial.begin(
```

```
  9600,
```

```
  SERIAL_8N1,
```

```
  GPS_RX,
```

```
  GPS_TX
```

```
);
```

```
Serial.println(  
    "GPS initialized."  
);
```

```
// =====  
// OLED  
// =====
```

```
Wire.begin(  
    OLED_SDA,  
    OLED_SCL  
);
```

```
if (  
    !display.begin(  
        SSD1306_SWITCHCAPVCC,  
        OLED_ADDRESS  
    )  
)  
{  
    Serial.println(  
        "ERROR: OLED not found!"  
    );
```

```
while (true)  
{  
    delay(1000);  
}
```

```
}
```

```
display.clearDisplay();
```

```
display.setTextColor(  
    SSD1306_WHITE  
);
```

```
display.setTextSize(1);
```

```
display.setCursor(15, 15);
```

```
display.println(  
    "GPS VEHICLE"  
);
```

```
display.setCursor(15, 30);
```

```
display.println(  
    "TRACKING"  
);
```

```
display.setCursor(15, 45);
```

```
display.println(  
    "STARTING..."  
);
```

```
display.display();
```

```
delay(2000);
```

```
// =====
```

```
// ADAFRUIT IO
```

```
// =====
```

```
connectAdafruitIO();
```

```
Serial.println();
```

```
Serial.println(  
    "System Ready!"
```

```
);
```

```
}
```

```
// =====
```

```
// 10. LOOP
```

```
// =====
```

```
void loop()
```

```
{
```

```
// =====
```

```
// MAINTAIN ADAFRUIT IO CONNECTION
```

```
// =====
```

```
io.run();
```

```
// =====
```

```
// READ GPS DATA
```

```
// =====
```

```
while (
    GPS_Serial.available()
)
{
    gps.encode(
        GPS_Serial.read()
    );
}

// =====
// CHECK UPDATE INTERVAL
// =====

unsigned long currentMillis =
    millis();

if (
    currentMillis -
    previousMillis >=
    UPDATE_INTERVAL
)
{
    previousMillis =
        currentMillis;

    // =====
    // CHECK GPS LOCATION
    // =====

    if (
```

```
gps.location.isValid()
)
{
double latitude =
    gps.location.lat();

double longitude =
    gps.location.lng();

double speed =
    gps.speed.kmph();

// =====
// SERIAL MONITOR
// =====

Serial.println();

Serial.println(
    "=====")
);

Serial.println(
    "    GPS VEHICLE DATA"
);

Serial.println(
    "=====")
);
```



```
Serial.print(  
  "Latitude : "  
);
```

```
Serial.println(  
  latitude,  
  6  
);
```

```
Serial.print(  
  "Longitude : "  
);
```

```
Serial.println(  
  longitude,  
  6  
);
```

```
Serial.print(  
  "Speed   : "  
);
```

```
Serial.print(  
  speed,  
  2  
);
```

```
Serial.println(  
  " km/h"  
);
```

```
Serial.print(  
  "Satellites: "  
);
```

```
Serial.println(  
  gps.satellites.value()  
);
```

```
Serial.println(  
  "GPS Status: ACTIVE"  
);
```

```
Serial.println(  
  "===== "  
);
```

```
// =====  
// OLED  
// =====
```

```
updateOLED(  
  latitude,  
  longitude,  
  speed,  
  true  
);
```

```
// =====  
// ADAFRUIT IO
```

```
// =====
```

```
latitudeFeed->save(  
  latitude  
);
```

```
longitudeFeed->save(  
  longitude  
);
```

```
speedFeed->save(  
  speed  
);
```

```
// Combined location
```

```
String location =  
  String(latitude, 6) +  
  "," +  
  String(longitude, 6);
```

```
locationFeed->save(  
  location  
);
```

```
statusFeed->save(  
  "Tracking Active"  
);
```

```
Serial.println(  
  "Data uploaded to Adafruit IO."
```

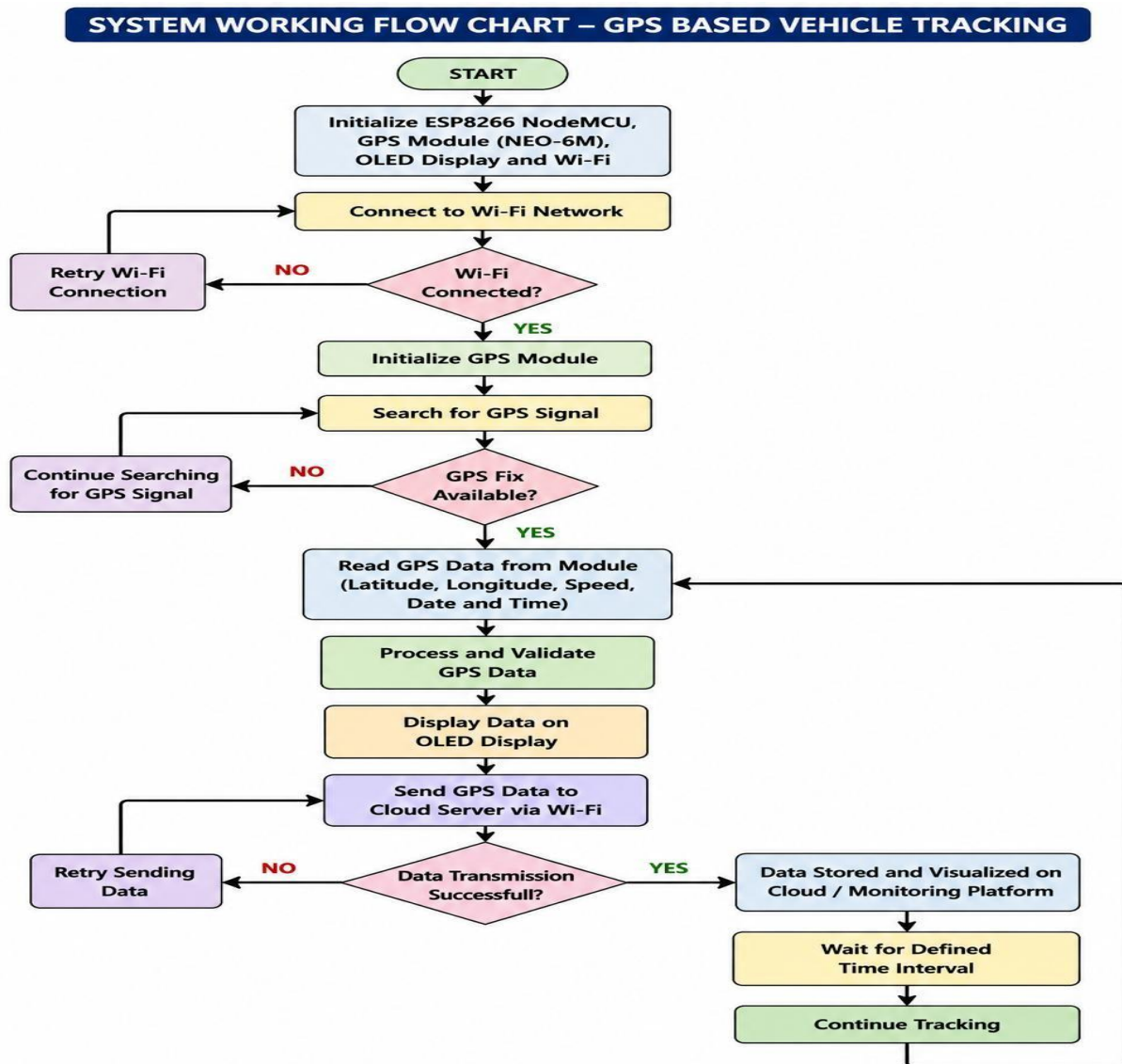
```
);  
}  
  
// =====  
// GPS NOT AVAILABLE  
// =====  
  
else  
{  
    Serial.println()  
  
    Serial.println(  
        "GPS signal not available."  
    );  
  
    Serial.print(  
        "Characters received: "  
    );  
  
    Serial.println(  
        gps.charsProcessed()  
    );  
  
    updateOLED(  
        0,  
        0,  
        0,  
        false  
    );  
}
```

```

statusFeed->save(
    "Waiting for GPS"
);
}
}
}

```

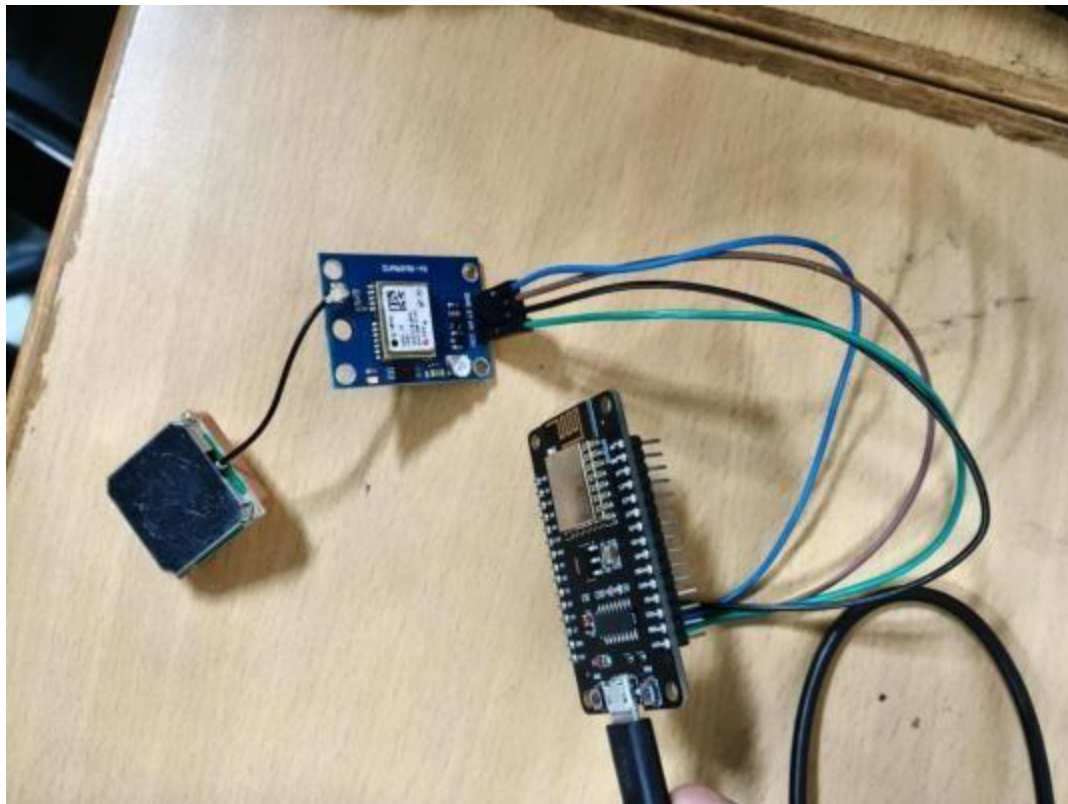
SYSTEM WORKING



BILL OF MATERIALS

Component	Quantity
ESP8266 NodeMCU	1
NEO-6M GPS Module	1
OLED Display	1
Li-ion Battery	1
TP4056 Charging Module	1
Jumper Wires	1 SET

PROTOTYPE IMPLEMENTATION



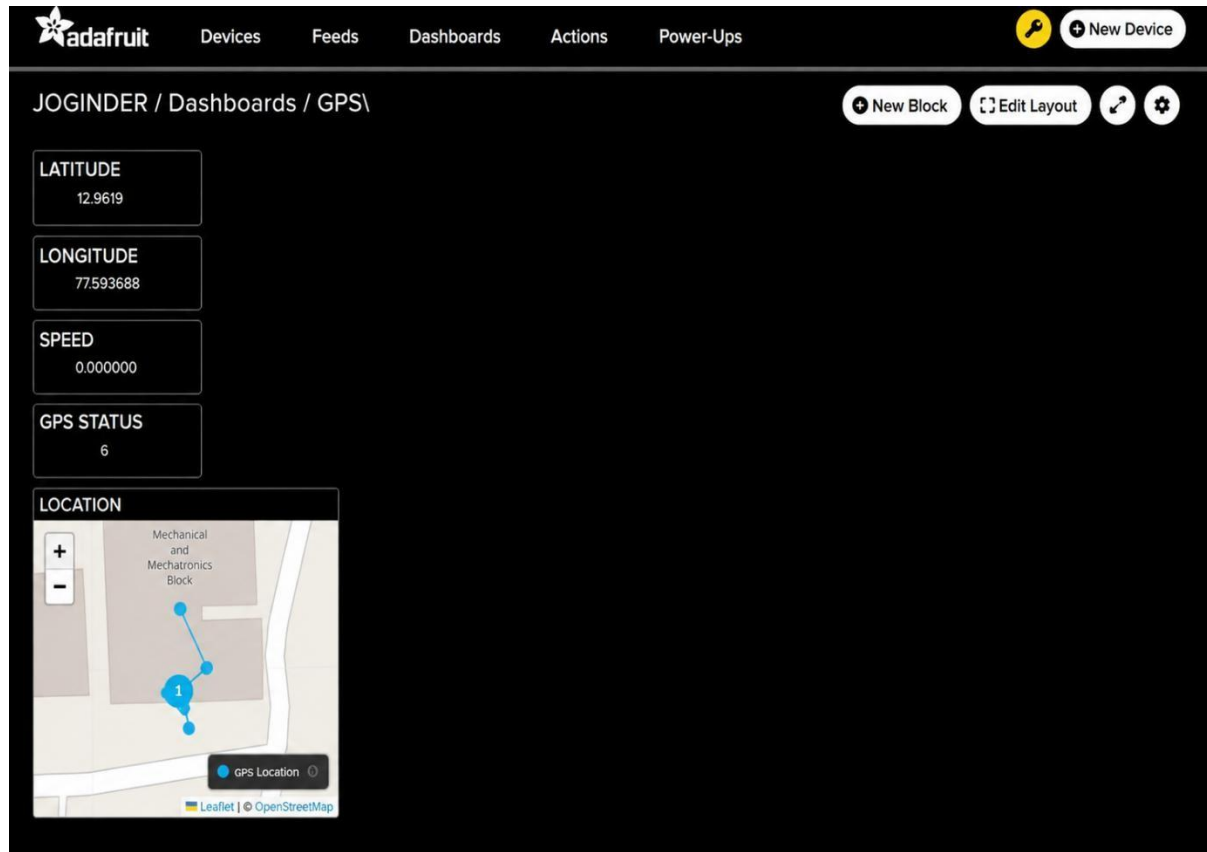
The prototype of the GPS-Based Vehicle Tracking and Fleet Management System is implemented using an ESP8266 NodeMCU, NEO-6M GPS module, OLED display, Li-ion battery, TP4056 charging module, and jumper wires. The ESP8266 acts as the main controller and coordinates the operation of the GPS module, display, and wireless communication.

The NEO-6M GPS module is connected to the ESP8266 through serial communication. It receives signals from GPS satellites and provides the vehicle's latitude, longitude, speed, date, and time. The ESP8266 processes this information and displays the tracking details on the OLED display using the I²C interface.

The built-in Wi-Fi capability of the ESP8266 is used to transmit the collected GPS information to a cloud server or fleet monitoring platform. The Li-ion battery provides portable power to the prototype, while the TP4056 module is used for battery charging. Jumper wires are used to establish the required electrical connections between the components.

After assembling the hardware, the ESP8266 is programmed to continuously receive and process GPS data. The prototype is tested by moving the tracking unit to different locations and observing the changes in GPS coordinates and speed. The collected data can then be transmitted to the monitoring platform, demonstrating the basic operation of real-time vehicle tracking and fleet management.

RESULT & IMPLEMENTATION ANALYSIS



The project can be evaluated using the following parameters.

1. GPS location

The system successfully obtains:

Latitude

Longitude when the GPS has a valid satellite fix.

2. Speed

The NEO-6M provides speed information, which is uploaded to the speed feed.

Example:

Speed = 32.5 km/h

3. Satellite count

The satellites feed indicates the number of satellites reported by the GPS.

Example:

Satellites = 7

A higher number generally indicates better GPS reception, although satellite count alone does not guarantee a particular accuracy.

4. Cloud connectivity

When Wi-Fi and internet access are available, the ESP8266 sends the data to Adafruit IO.

5. Map tracking

The location feed can be used to display the vehicle position on the Adafruit IO Map.

6. Upload interval

The proposed code uploads information approximately every 10 seconds.

7. Indoor performance

A closed room can reduce GPS signal reception. In this situation:

Wi-Fi → May work normally

ESP8266 → Works normally

Adafruit IO → Can work normally

GPS → May not obtain a valid fix

Therefore, the GPS antenna should preferably have a clear view of the sky.